Aula 34: Formulários e Validação em React

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

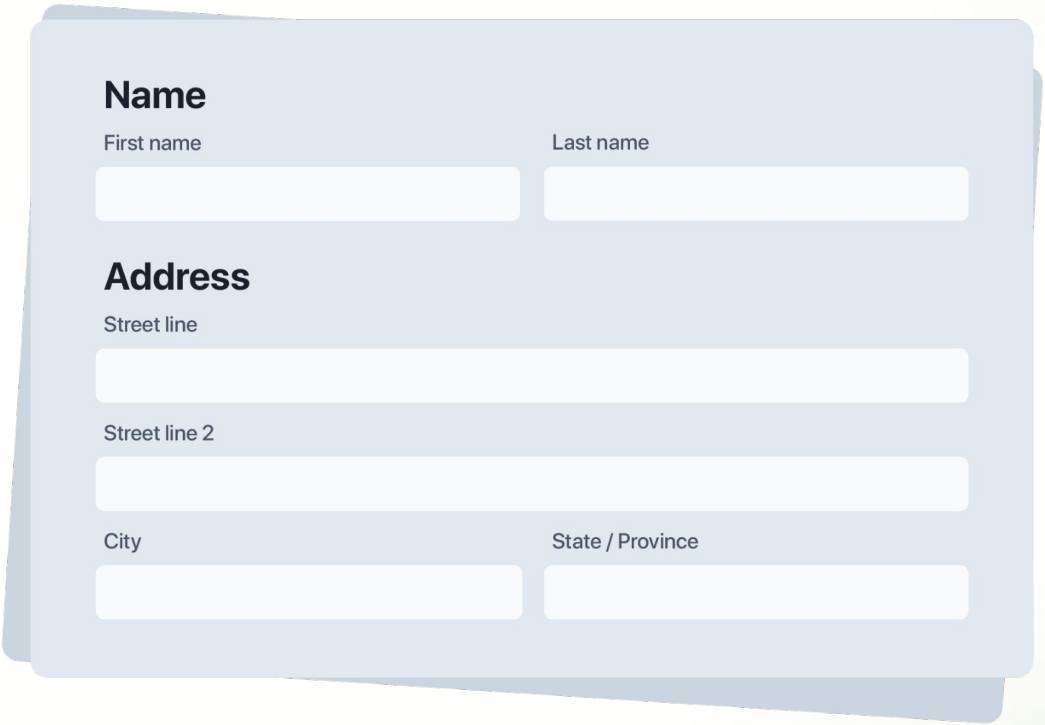
Índice

- INTRODUÇÃO
- FORMULÁRIO EM REACT
- INPUTS CONTROLADOS
- O PROBLEMA DA ESCALABILIDADE
- UNCONTROLLED COMPONENTS & REFS
- VALIDAÇÃO MANUAL E FEEDBACK AO USUÁRIO
- VALIDAÇÃO COM BIBLIOTECAS
- REACT HOOK FORM (RHF)
- VALIDAÇÃO COM ZOD
- FORMULÁRIOS COMPLEXOS
- EXERCÍCIOS

Introdução

IMPORTÂNCIA

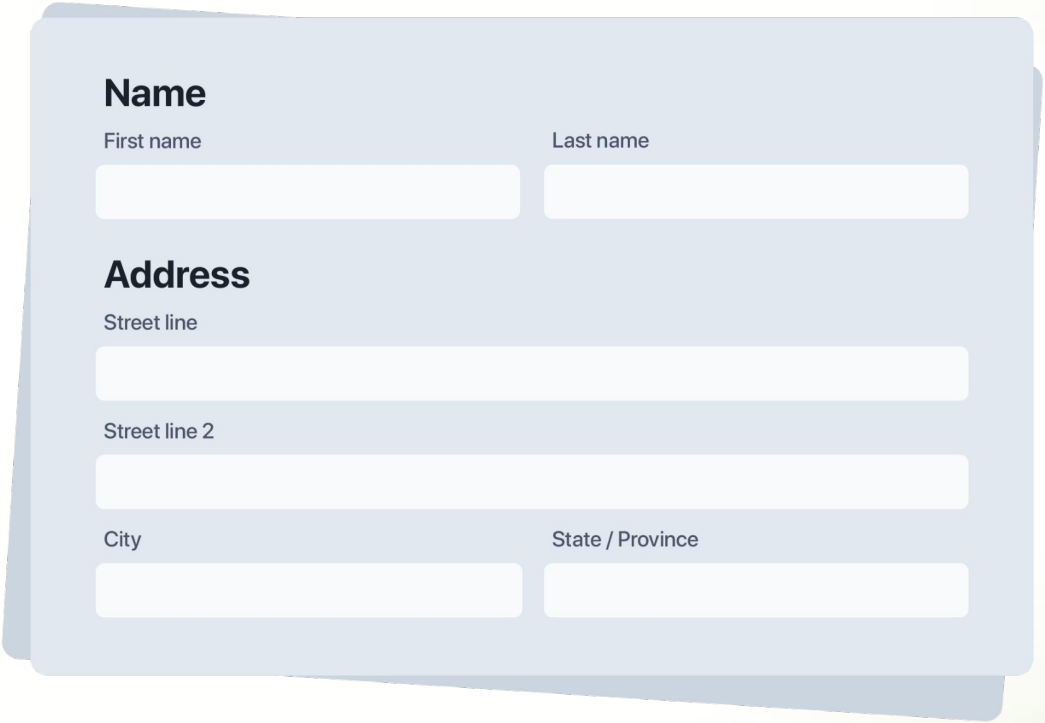
- Principal forma de interação usuário ↔ aplicação
- Login, cadastro, pagamento, busca, configurações
- Fonte comum de bugs e má experiência do usuário



Introdução

DESAFIOS

- Controle de estado
- Validação correta dos dados
- Mensagens de erro claras
- Escalabilidade do código



Formulários em React

- HTML tradicional: DOM controla o estado
- React: estado controlado pelo componente
- Inputs controlados
  - Valor do input vem do state
  - Alterações via onChange

Inputs Controlados

- Formulário com componentes controlados são o exemplo mais comum inclusive em bibliotecas
- Nessa abordagem a cada mudança no valor do input uma função é chamada para atualizar o valor que é salvo no estado do componente
- Exemplo: Input de texto
  - value={state}
  - onChange={(e) => setState(e.target.value)}

Inputs Controlados

- Tipos comuns de inputs
  - text
  - email
  - password
  - checkbox
  - radio
  - select
  - textarea

Inputs Controlados

- Boas práticas
  - Nomear corretamente os campos
  - Evitar lógica complexa dentro do JSX

Inputs Controlados

```
function LoginForm() {
  const [email, setEmail] = useState("");

  return (
    <form>
      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <p>O email é: {email}</p>
    </form>
  );
}
```

Exercício Prático

FORMULÁRIO DE CADASTRO SIMPLES

- Crie um formulário com os seguintes campos:
  - Nome
  - Email
  - Senha
  - Botão "Cadastrar"
- Todos os campos controlados
- Exibir os valores no console ao enviar

O problema de escalabilidade

- E se o formulário tiver 20 campos?
- Problemas:
  - Excesso de useState
  - Múltiplas funções de onChange
  - Boilerplate excessivo
  - Renderizações desnecessárias a cada tecla digitada

O problema de escalabilidade

- Uma solução é o uso de Uncontrolled Components & Refs
- Conceito:
  - Sincronizar cada tecla digitada com o estado do React causa re-renders em todo o formulário
  - Ao invés disso os valores são acessados apenas quando necessário (no submit)

Uncontrolled Components & Refs

- A Ferramenta:
  - O hook useRef
- Possui vantagem de performance máxima
  - O campo de input retém o valor nativamente no DOM, e o React não "sofre" a cada letra nova
- É usado em formulários muito simples ou quando se busca por performance extrema

Uncontrolled Components & Refs

CARACTERÍSTICA	CONTROLADOS (USESTATE)	NÃO CONTROLADOS (USEREF)
Fonte da Verdade	Estado do React	DOM (Navegador)
Performance	Re-renderiza a cada tecla	Mais performático (menos renders)
Validação	Instantânea e fácil	Geralmente na submissão
Complexidade	Mais código repetitivo	Código mais limpo

Validação e Feedback ao Usuário

- Validação:
  - Garantir que os dados estejam corretos antes do envio
- Tipos de validação
  - Obrigatoriedade
  - Tamanho mínimo/máximo
  - Formato (email, senha forte)

Validação e Feedback ao Usuário

- Onde validar?
  - No onSubmit
  - Em tempo real (onBlur, onChange)
- Exemplo Conceitual
  - Verificar campos vazios
  - Criar objeto de erros
  - Atualizar estado de erros

Validação e Feedback ao Usuário

- Exemplo de erros
  - Nome obrigatório
  - Email inválido
  - Senha com menos de 6 caracteres

Validação e Feedback ao Usuário

- Importância
  - Reduz frustração
  - Guia o usuário
- Tipos de feedback
  - Mensagens de erro abaixo do campo
  - Bordas coloridas
  - Mensagens globais
  - Desabilitar botão de envio

Validação e Feedback ao Usuário

- Experiência do Usuário (UX):
  - Não espere o envio para mostrar erros óbvios
  - Mensagens de erro claras e acessíveis (ARIA labels)
  - Estados visuais: error, success, loading
- Tipos de Validação: Presença, Formato (Regex), Comprimento, Correspondência (Senha)

Validação e Feedback ao Usuário

- Experiência do Usuário (UX):
  - Não espere o envio para mostrar erros óbvios
  - Mensagens de erro claras e acessíveis (ARIA labels)
  - Estados visuais: error, success, loading
- Tipos de Validação: Presença, Formato (Regex), Comprimento, Correspondência (Senha)

Validação com Bibliotecas

- Por que usar bibliotecas?
  - Menos código repetitivo
  - Melhor organização
  - Escala melhor para formulários grandes

Validação com Bibliotecas

- Bibliotecas populares
  - React Hook Form
  - Formik
  - Yup
  - Zod

React Hook Form (RHF)

- Pode ser considerado como a solução definitiva para o problema da escalabilidade
- O RHF utiliza internamente Uncontrolled Components de forma inteligente
- Características
  - Baseado em hooks
  - Melhor performance
  - Menos re-renderizações

React Hook Form (RHF)

- Como ele resolve a escalabilidade?
  - Isolamento de Renderização:
    - Apenas o componente que está sendo validado ou alterado renderiza, não o formulário inteiro
  - Menos Código (Boilerplate):
    - Não é criado um useState para cada campo
    - Apenas "registra" o input

React Hook Form (RHF)

- Como ele resolve a escalabilidade?
  - Inscrição (Subscription):
    - É possível "observar" (watch) campos específicos sem afetar os outros
  - O RHF resolve os problemas de performance e verbosidade usando componentes não controlados por baixo dos panos
  - Mas com uma API de hooks amigável

React Hook Form (RHF)

- Performance (menos re-renders) e simplicidade.
- Conceitos Chave:
  - register: Conecta o input ao hook
  - handleSubmit: Trata o envio de forma segura
  - errors: Objeto que contém os erros de validação
- Instalação:

```
npm install react-hook-form
```

React Hook Form (RHF)

```
import { useForm } from "react-hook-form";

function SimpleForm() {
  const { register, handleSubmit } = useForm();
  const onSubmit = (data) => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("firstName")} placeholder="Nome" />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

React Hook Form (RHF)

```
import { useForm } from "react-hook-form";

function SimpleForm() {
  const { register, handleSubmit } = useForm();
  const onSubmit = (data) => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("firstName")} placeholder="Nome" />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

O hook **useForm** fornece as ferramentas necessárias, enquanto o **register** conecta os inputs ao estado do formulário de forma eficiente.

Validação com Zod

VALIDAÇÃO POR ESQUEMA

- Em vez de validar cada campo manualmente com condicionais complexas, define-se um esquema de dados que descreve a estrutura e as regras esperadas
- **Segurança de Tipos**: Inferência automática de tipos TypeScript.
- **Centralização**: Regras de negócio em um único lugar.
- **Legibilidade**: Código mais limpo e declarativo.
- **Flexibilidade**: Mensagens de erro customizadas por campo.

Validação com Zod

VALIDAÇÃO POR ESQUEMA

- Zod também precisa ser instalado: npm install zod

```
const schema = z.object({
  email: z.string().email(),
  age: z.number().min(18)
});
```

Integrando Zod ao RHF

1. DEFINA O ESQUEMA

```
const schema = z.object({
  email: z.string()
    .email("Email inválido"),
  password: z.string()
    .min(8, "Mínimo 8
caracteres"),
});
```

2. CONECTE O RESOLVER

```
const {
  register,
  handleSubmit,
  formState: { errors }
} = useForm({
  resolver: zodResolver(schema)
});
```

O **zodResolver** traduz os erros do Zod para o formato que o React Hook Form entende automaticamente.

Exercício Exemplo

- Formulário com React Hook Form
- Criar um formulário de login com:
  - Email
  - Senha
- Requisitos:
  - Usar React Hook Form
  - Validação com Yup ou Zod
  - Exibir mensagens de erro automáticas

Formulários Complexos

- Exemplos
  - Múltiplas etapas (wizard)
  - Campos condicionais
  - Listas dinâmicas de inputs

Formulários Complexos

- Estratégias
  - Componentizar formulários
  - Separar lógica de validação
  - Reutilizar schemas

Exercício

- Objetivo: Cadastro de Evento Profissional
- Crie um formulário de cadastro que contenha:

1. Nome Completo (Mínimo 3 caracteres).
2. E-mail (Validação de formato).
3. Tipo de Ingresso (Select: VIP, Comum, Estudante).
4. Termos de Uso (Checkbox obrigatório).

Exercício

- Objetivo: Cadastro de Evento Profissional
- Requisitos:
  - O botão de "Enviar" deve começar desabilitado
  - Exibir mensagem de erro em vermelho abaixo de cada campo inválido
  - Ao enviar, exibir um alert com o JSON dos dados e limpar o formulário

Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Formulários e Validação**
  - Formulário para criação/edição de tarefas
  - Controle de inputs
  - Validação de campos obrigatórios
  - Feedback visual para erros
- Funcionalidade: Criar páginas separadas

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Formulários e Validação**
- O que será feito:
  - Criar uma página ou modal específico para adicionar tarefa.
  - Campos: Título, Descrição (textarea) e Categoria (select: Trabalho, Pessoal, Estudo).
  - Validação: Impedir criar tarefa sem título ou com título muito curto (< 3 letras).
  - Exibir mensagem de erro vermelha abaixo do input.
- Entrega: O botão "Salvar" fica desabilitado ou mostra erro se o usuário tentar enviar dados vazios.